

(20 pts) Questão 1: Um medidor de nível possui 4 leds indicadores e um alarme, como mostrado na imagem abaixo. No total são 4 leds, conectados do bit 0 ao bit 3 da porta A e um dispositivo de alarme conectado no bit 4 da porta A que dispara quando o nível ultrapassa 100%.



Monte uma **biblioteca** de interface para esse painel com 3 funções:

- void painelInit(void); // configura o sistema
- void painelAlarme(int status); // liga ou desliga o alarme
- void painel nivel(int status); // Ajusta o nível mostrado

(25 pts) Questão 2: Um motor controlado por sinal PWM possui um circuito de controle microprocessado com um potenciômetro de ajuste ligado ao seu conversor AD. Quando a tensão no potenciômetro atinge 3,3V o conversor AD fornece o seu valor máximo (1023). Quando o sinal PWM é igual a zero o motor está parado, quando o sinal PWM é igual a 100% o motor está na sua maior velocidade. Uma chave (bit 3 PORTA) desativa o controle por potenciômetro colocando o motor na sua velocidade máxima. Implemente um programa para controlar este motor.

(25 pts) Questão 3: Um contador universal utilizado na indústria mostra em um display de LCD o número de peças na linha de montagem. Além do display, este dispositivo envia essa informação para uma porta serial podendo ser integrado a outros sistemas. A contagem é representada por 4 dígitos no display de LCD sendo a mesma informação enviada para a porta serial. Uma chave (bit 0 PORTA) sinaliza ao microcontrolador a passagem de uma peça. Implemente um programa para esse dispositivo.

(30 pts) Questão 4: Um placar eletrônico mostra o tempo de jogo em um display de 7 seguimentos com dois dígitos (somente minutos). Os outros dois dígitos são utilizados para registrar os gols de cada time, um para cada time (não marca goleada). O sistema deve ter um botão que permite zerar o tempo (bit 0 PORTA) e dois botões para incrementar os gols do time A e do time B (bits 1 e 2 PORTA). Implemente o **programa** utilizando a interrupção do Timer para controlar o tempo do relógio.

```
char flag;
//Esta interrupção é chamada cada 1ms.
void Int(void) __interrupt(){
    if (BitTst(INTCON, 2)) { //Verifica se ocorreu overflow de TMR0
        timerReset(1000); //Ajusta o contador do TMR0
        BitClr(INTCON, 2); //Desliga o Flag de everflow do TMR0
        flag = 1;
    }
}
```

Obs: Uma função InitInterrupt() ajusta todos os parâmetros da interrupção.

```
//adc.h
void adcInit(void);
int adcRead(void);
```

```
//ldc.h
void ldcInit(void);
void lcdChar(char letra);
void lcdCommand(char val);
```

```
//pwm.h
void pwmInit(void);
void pwmFrequency(unsigned int freq);
void pwmSet1(unsigned char por cento);
```

```
//config.h
Necessário para configurar o hardware
Possui comandos: #pragma ...
```

```
//pic18f4520.h
Define TRISA, PORTA,... bem como:
BitSet(arg,bit) BitClr(arg,bit)
BitFlp(arg,bit) BitTst(arg,bit)
```

```
//serial.h
void serialInit(void);
unsigned char serialRead(void);
void serialSend(unsigned char c);
```

```
//ssd.h
void ssdInit(void);
void ssdUpdate(void);
void ssdDigit(int val, int pos);
```

```
//Observação: TRIS = 0 → Saída
//Em ASCII caracter '0' = 48
```